Assignment in
The Finite Element Method, 2026

Division of Solid Mechanics

The task is to write a finite element program to analyze temperature and stress distributions in an electric car component, made of structural batteries. The problem should be solved with MATLAB or Python with the aid of CALFEM.

Problem description

The electric car industry faces increasing pressure to increase battery efficiency while maintaining strict safety and durability standards. One proposed solution is structural batteries: composite systems designed to both carry mechanical loads and store electrical energy. This has the potential of reducing overall system mass by eliminating separate battery units. However, due to the heat generated by the batteries, it remains a challenge to design a durable structure able to withstand both mechanical and thermal loads.

As such, computational engineers (you) have been assigned with analyzing the thermo-mechanical behavior of a new electric car component, whose geometry is shown in Fig. 1a. The structure contains cooling channels, represented by circles. The coordinates for the centres of these circles are given in Table 1.

Your analysis is divided into two parts. First, consider the thermal behavior. During operation, the battery generates heat, which is represented by the body heat Q. Due to natural convection at the top boundary and the cooling channels, a small portion of the applied heat is radiated into the surrounding environment. For this convection, assume $\alpha_c = 50\ W/m^2K$ and $T_\infty = 293°K$. Further, the component is subject to a cooling heat flux q_{out}, and its initial temperature is $T_0 = 293°K$. The corresponding boundary conditions are shown in Fig. 1b.

Next, consider the mechanical behavior. Here, the component is assumed to be subject to two distributed loads $p_2 = -10^7$ Pa (compressive) and $p_1(y) = p_0(\frac{y}{0.023} - 1)$ (compressive and tensile) with $p_0 = 10^8$ Pa. To facilitate the cooling, the cooling channels are pressurized, which exerts a pressure $p_c = 10^6$ Pa on the component. Assume plane strain.

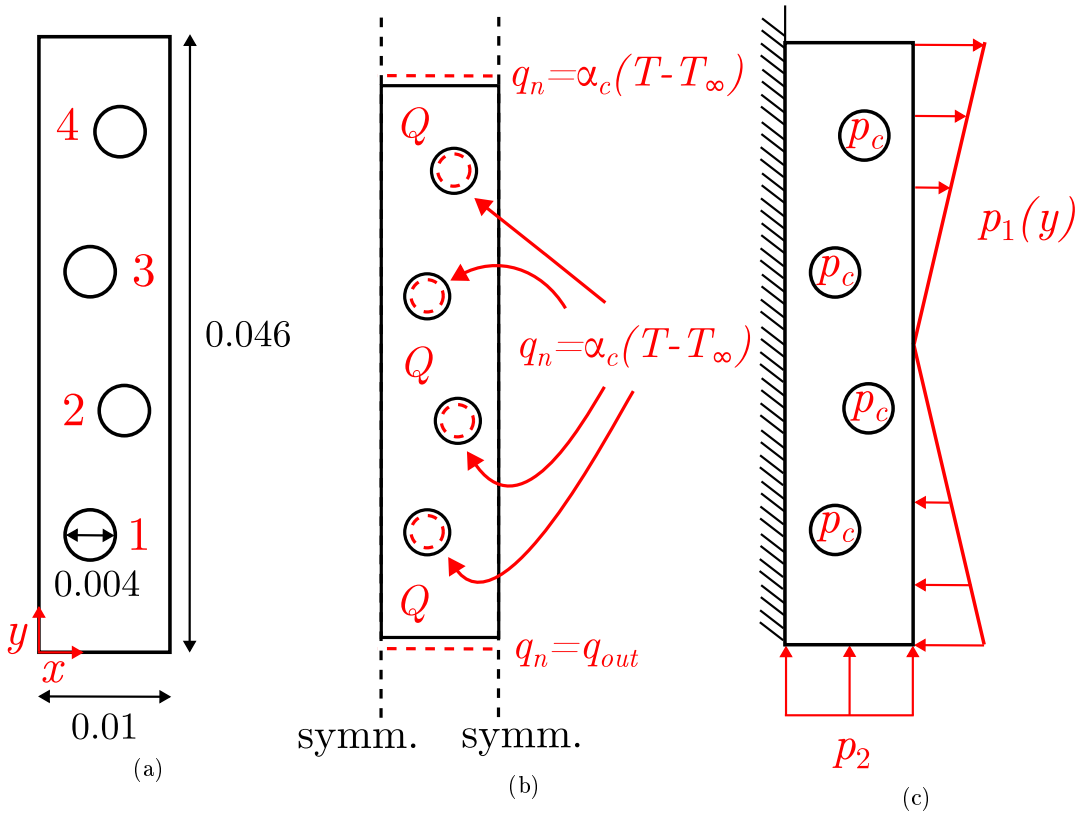

Figure 1: Diagrams of a cross section of the electric car component. a) Geometry, with measurements given in m, and the out-of-plane thickness is 1 m. The coordinates for the centres of the circles 1-4 are given in Table 1. b) Boundary conditions for the thermal problem, where "symm" denotes a vertical line of symmetry. c) Boundary conditions for the mechanical problem.

	Coordinates (m)
Circle 1	(0.004,0.008)
Circle 2	(0.006,0.018)
Circle 3	(0.004,0.028)
Circle 4	(0.006,0.038)

Table 1: Coordinates (x, y) for the centres of the circles 1-4 in Fig. 1a.

The material is modelled as homogeneous, with the material parameters in Table 2.

	Value
Young's modulus, E [GPa]	30
Poisson's ratio, ν [-]	0.3
Expansion coefficient, α [$1/K$]	$8.0{\cdot}10^{-6}$
Density, ρ [kg/m^3]	1500
Specific heat, c_p [J/kgK]	800
Thermal conductivity, k [W/mK]	2.0

Table 2: Material data.

Tasks

The tasks to be considered are given below. Note that you can still pass the assignment if you replace the convective boundary conditions in task a) and b) with the boundary condition $T = T_0$. However, this means you can obtain a maximum of 3 points for the assignment.

a) Compute the stationary temperature distribution in the component. Assuming the constant body heat $Q = 4 \cdot 10^5$ W/m^3 and heat flux $q_{out} = 2\ 000\ W/m^2$, what is the maximum temperature in the component, and which parts of the component reach the highest temperatures?

b) Compute the transient temperature distribution during 10 minutes of applied heat. For this, assume a time dependent body heat $Q(t) = Q_{max} \sin\left(\frac{\pi}{t_{tot}} t\right)$ and heat flux $q_{out}(t) = q_{max} \sin\left(\frac{\pi}{t_{tot}} t\right)$ where $Q_{max} = 4 \cdot 10^5$ W/m^3, $q_{max} = 2\ 000\ W/m^2$ and the total simulated time is $t_{tot} = 10$ min. Plot the maximum temperature in the component over time. Also plot the temperature distribution in the component at 6 different time steps. How does the temperature distribution change over time?

c) For the mechanical problem, at the boundaries where pressure is applied, plot the surface normals to ensure that they are computed correctly. Compute the displacement field due to the stationary thermal strains. Plot the von Mises stress distribution in the component when stationary thermal strains are present. If the component starts to yield at $\sigma_{eff} = 150$ MPa, do you expect the component to withstand the 10 minutes of applied heat? What problems can you identify with the design of the component? What problems can you identify with the modelling assumptions?

Hint: The von Mises stress is defined as:

$$\sigma_{eff} = \sqrt{\sigma_{xx}^2 + \sigma_{yy}^2 + \sigma_{zz}^2 - \sigma_{xx}\sigma_{yy} - \sigma_{xx}\sigma_{zz} - \sigma_{yy}\sigma_{zz} + 3\tau_{xy}^2 + 3\tau_{xz}^2 + 3\tau_{yz}^2}$$

where it is important to not forget the out-of-plane stress component due to the assumption of plane strain.

Details

Triangular, 3-node elements are suitable for this task. A fully implicit time integration scheme, i.e. $\theta = 1$, should be used. Note that the element function for forming, $\boldsymbol{C}^e$, is available on the course homepage. As a starting point, use the strong formulations of the heat and the mechanical problems. Also note that you must derive the contributions for the convection boundary condition yourselves.

Report

A fundamental ingredient in all research is that it should be possible to regenerate the results obtained based on the report. In the present situation this implies that the appended source code should only be considered as supporting material. Moreover, note that one variable for grading the report is the structure of the computer code, i.e. you should choose suitable names for variables. A suitable structure for the report is:

- **Introduction**: Description of the problem, geometry and boundary conditions.
- **Procedure**: Present the governing PDE on weak or strong form, as well as the spatial and time discretizations you have used. Note that you are encouraged to make references to textbooks. Though, it is important to carefully present all calculations that are not available in the literature.

- **Results**: Present the results in illustrative figures and/or tables. Note that the results should be commented such that the reader can not misunderstand the results (correct labels, units, figure texts etc.)
- **Discussion**: A discussion of the results, their meaning and significance. You might want to discuss sources of errors and accuracy in this section.
- **Computer Code**: Note that the code should be easy to follow and all declared variables should have intuitive names and be well commented.

A well structured report in **English** should be handed to the Division of Solid Mechanics no later than **2026-05-24**. Source files should be well commented and submitted alongside the report. The reader is assumed to have the same knowledge level as the author. If the report contains major programming or theoretical errors, the report is returned in order to be corrected. It is possible to obtain up to 4 points which are added to the points obtained at the exam in June 2026. The assignment should be approved no later than **2026-06-09**. You should submit your report in **PDF** format in Canvas. Note that the bonus points obtained is only valid for the examination in June 2026.

Collaboration

The task should be solved in groups of *two*.

Usage of generative AI

The project report must include a brief Generative AI (GAI) usage disclosure, stating:

- Which tool(s) were used. It should also be stated if no tool were used.
- For what purpose GAI was used.
- Parts of the work that were affected and how.

An example: "ChatGPT was used for improving clarity and grammar of the written text, and for occasionally debugging code. All final content was understood and verified by the student(s)."

Responsibility and Accountability: Students are fully responsible for the correctness, originality, and explainability of all submitted work, including AI-assisted content. Inability to explain submitted work can result in penalties.

Academic Integrity: Improper or undisclosed use of GAI will be handled in accordance with the institution's academic integrity policy and may result in disciplinary action, including warnings or suspension.

Generating mesh in python

The mesh should be generated using GMSH, which is wrapped in calfem. We recommend you follow the examples found in the documentation.

Generating mesh in MATLAB

In this assignment you should create your own mesh using the built-in PDEtool in MATLAB. This tutorial considers the example geometry seen in figure 2, which of course must be changed for your specific application.

PDEtool can be used directly to create simple geometries with little effort. To start just type `pdetool` into the MATLAB terminal to open PDEtool user interface. Although the tool is originally designed for solving partial differential equations only the meshing aspect are of interest in this assignment.

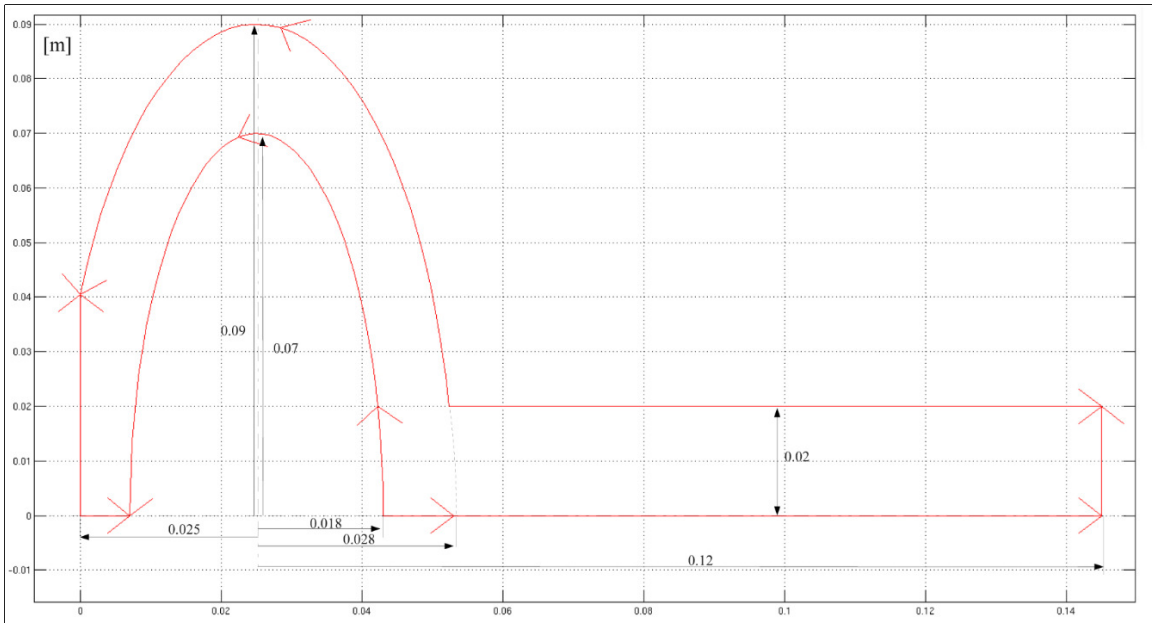

Figure 2: Dog toy geometry.

A step by step tutorial of how to use some of the basic features of PDEtool are provided here. Before starting to draw it is often convenient to activate grid and change the axis scales. This is done using <Options> menu. To build our geometry simple geometries are added one by one. The basic shapes used in PDEtool are rectangles, ellipses and polygons. Basic drawing tools can be found on the toolbar (see figure 3, note polygons are not necessary to create the geometry for the lab).

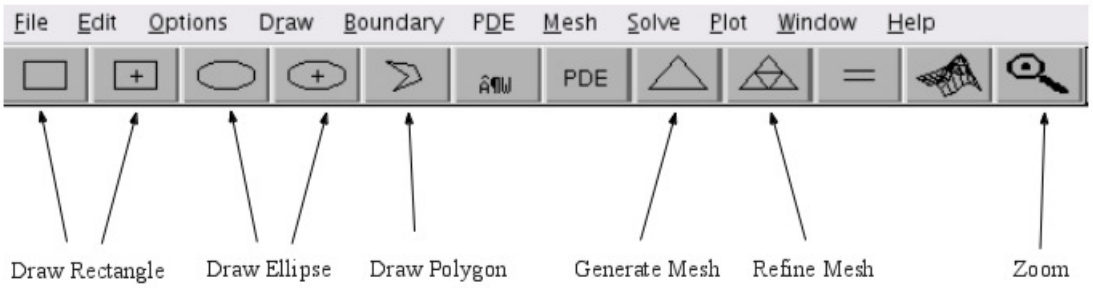

Figure 3: Basic tools for drawing in PDEtool.

Draw a rectangle: Create a rectangle by using the rectangle tool (see figure 3) and simply hold left mouse button and move the mouse cursor to obtain the desired size. To adjust the size and position double click on the rectangle and a dialog will appear where coordinates and dimensions can be provided, see figure 4.

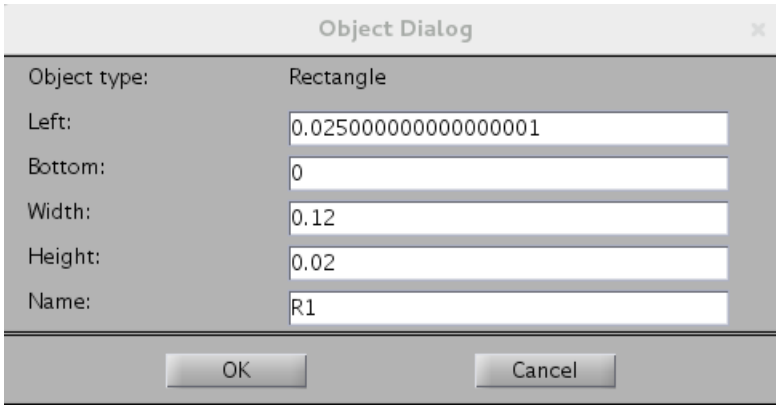

Figure 4: Rectangle object dialog.

Draw an ellipse: Create a ellipse by using the draw ellipse tool (see figure 3) and simply hold left mouse button and move the mouse cursor to obtain the desired size. Double click to adjust position and size in similar fashion as for rectangles.

Determining combined geometry: In the box below shape-buttons the present features are presented by name (which may be edited using object dialog). The plus sign indicates that geometries are added and minus signs indicate that they are cut out. Because of this the order is important i.e **R1+E1-E2 ≠ R1-E2+E1 in general**. Below figues showing drawn features and the resulting body after combining by grouping and changing signs in the set formula row (see figure 5, 7 and 6). Note that to see the resulting geometry we need to go to either meshing or boundary view (see toolbar).

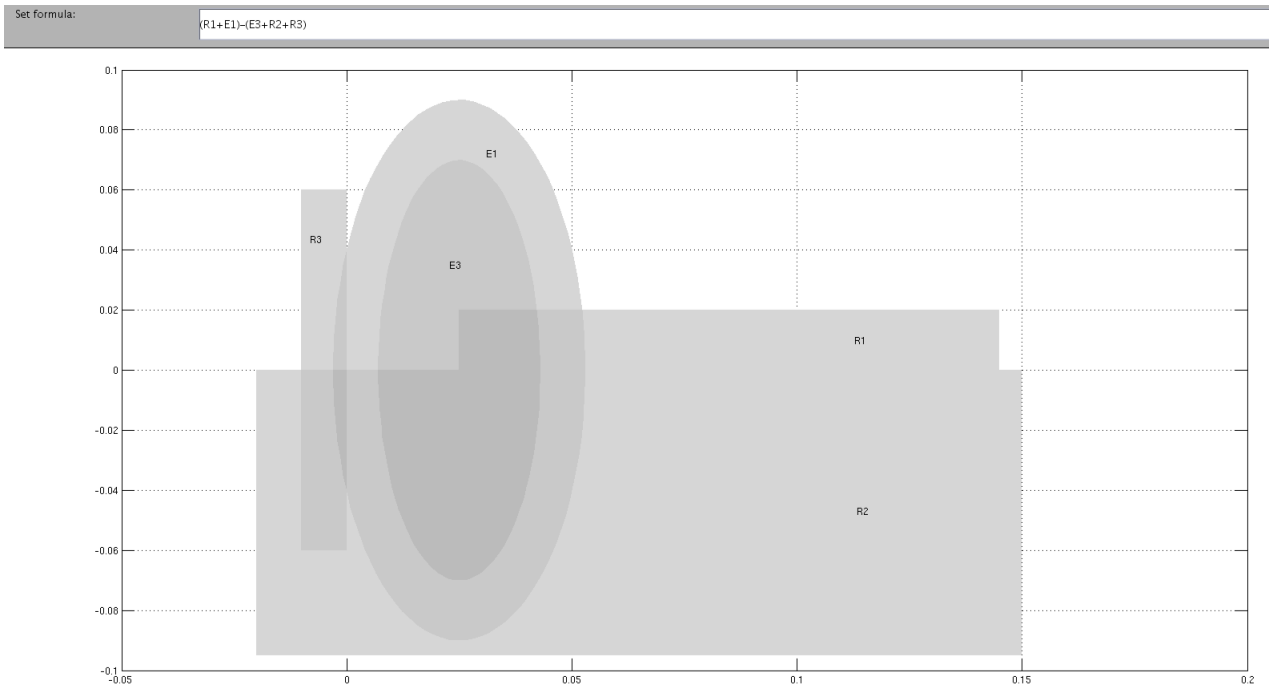

Figure 5: Set of combined shapes.

When the necessary features are added go to <Boundary> / <Boundary mode> to see which are the boundaries of your geometry. The red arrows define the main borders and the grey contours represent the so called subdomains. For instance, subdomains appear where features overlap. To

get a uniform mesh it is convenient to remove the subdomains. To do so use <Boundary> / <Remove all Subdomain Borders>. If the mesh consist of different natural domains such as different materials it is favourable to keep the subdomains.

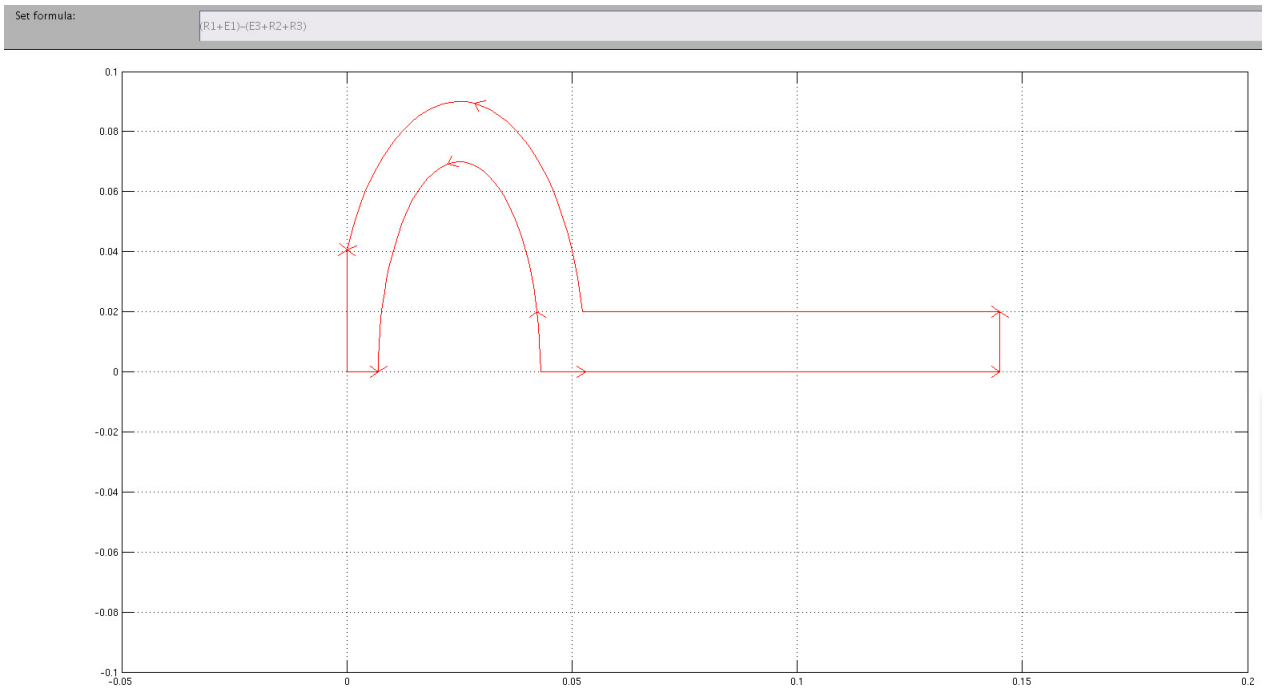

Figure 6: Resulting geometry seen in boundary mode (ctrl+B) after all subdomains have been removed.

To obtain the geometry provided in figure 6 the formula used is given in figure 7 and the object names is seen in figure 5.

Figure 7: Formula determining active areas.

To generate a mesh simply press mesh tool when the geometry seen in boundary mode is satisfactory (see figure 3). To refine the mesh, i.e. create more elements, press refine mesh until enough elements are generated (remember that finer mesh means higher computational cost but better accuracy). Note that a high number of elements increases the computational time.

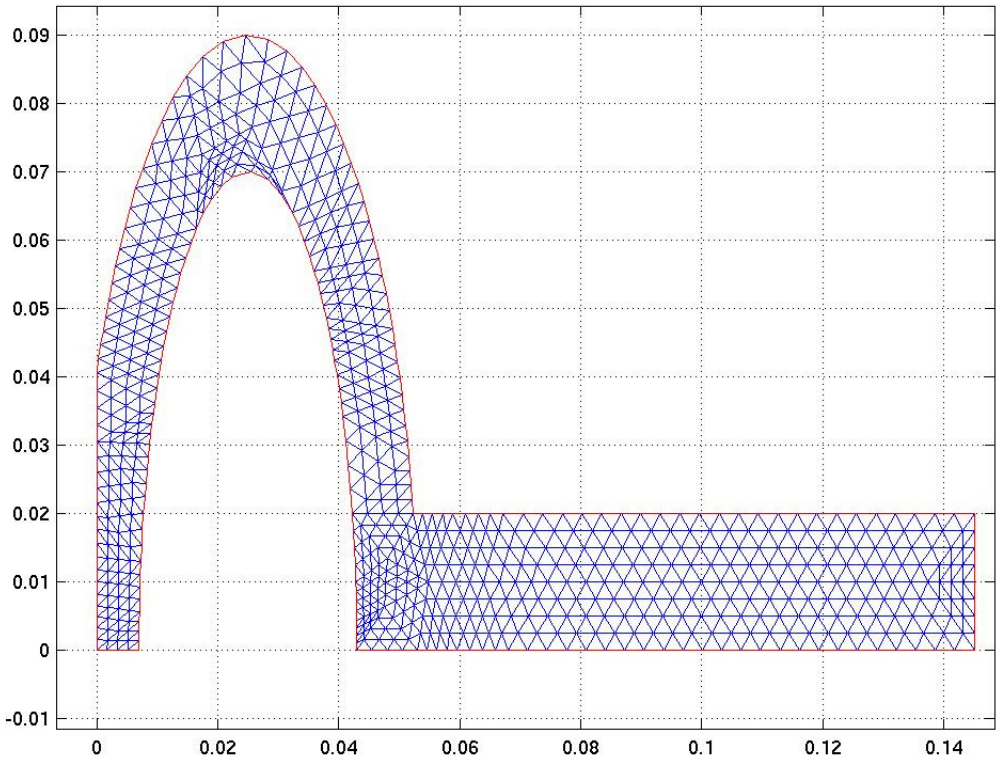

Figure 8: Mesh after two refinements.

When you are content with your mesh use <Mesh> / <Export mesh> to save the topology matrices associated with the current mesh (note you have to save the work separately to keep geometries etc. since <Export mesh> will only save matrices). The topology matrices are `p,e,t` (points, edges, triangles). The exported matrices will directly appear in the active MATLAB workspace. It is strongly advised to save them directly in a `.mat` file (mark variables in workspace, right click and save) so it can be loaded in the scrips you write. From `p,e,t` the CALFEM quantities `edof`, `dof`, `coord` etc can be extracted. On the course web page for FEM FAQ are some instructions of how this is done.

General tips for python

For general tips regarding calfem, see the documentation.

You may need to manually specify the backend for calfem's visualization engine, to do so add the following code at the top of your python script.

```python
# IMPORTS
import calfem.vis_mpl as cfv
import matplotlib as mpl
mpl.use('TkAgg')
# YOUR PROGRAM
```

From the meshing module in calfem you will recieve a dictionary of boundary markers and the nodes in the finite element mesh which lie on them. To compute intergrals over the boundary looping over a list of edges is more convenient. The function below can be used for this.

```python
def nodesToEdges(nodes: dict, enod: np.array) -> dict:
  """ Returns a list of edges given nodes

  Args:
  nodes (np.array): nodes on boundary
  enod (np.array): element connectivity matrix

  Returns:
  list: edges on boundary
  """
  # Initialize edges dict
  edges = {}
  for key in nodes.keys():
              edges[key] = []

  for con in zip(enod):
    for key in nodes.keys():
      I = np.intersect1d(con, nodes[key])
      if len(I) == 2:
        edges[key].append(I)

  return edges
```

Since stresses are computed on the gaussian points / centers of elements, you will have an element-wise constant effective stress field. To estimate a node-based effective stress field, and hence get an element-wise linear approximation to the effective stress, the function below can be used.

```python
def elmToNode(eV: np.array, edof: np.array) -> np.array:
  """ Estimates nodal values from element-based values

  Args:
  eV (np.array): element values
  edof (np.array): element connectivity matrix

  Returns:
  np.array: nodal-based values
  """

  nnod: int = np.max(edof)
  ne: int = 0
  nV = np.zeros((nnod,))
  # Loop over nodes
  for n in range(0, nnod):
    ne = 0
    # Check which elements contain the node
    for e, eldof in enumerate(edof):
      # If e contains the node add the elemental value
      if ((n+1) in eldof):
        ne += 1
        nV[n] += eV[e]
```

```
    # Divide by total number of elements
    nV[n] /= ne
  return nV
```

For plotting, we reccomend that you use the module calfem.visualization.

General tips for MATLAB

The output data from PDEtool is in the form `p,e,t` where `p` is *points*, it is a 2 row matrix where the first row is the x-coordinates and the second row is the y-coordinates. There are one column for each node i.e. in CALFEM notation `coord = p'`.

The output `t` is *triangles* and it has dimensions $4 \times$ nelm. The three first rows are the node numbers associated with each element (three nodes for linear triangular elements) while the fourth row is its subdomain. The subdomain is great to use when you want to associate specific material properties to each element. Rows 1,2 and 3 can be used to calculate our `edof` matrices, as an example

```
enod=t(1:3,:)';                        % nodes of elements
nelm=size(enod,1);                     % number of elements
nnod=size(coord,1);                    % number of nodes
dof=(1:nnod)';                         % dof number is node number
dof_S=[(1:nnod)',(nnod+1:2*nnod)'];    % give each dof a number
for ie=1:nelm
edof_S(ie,:)=[ie dof_S(enod(ie,1),:), dof_S(enod(ie,2),:),dof_S(enod(ie,3),:)];
edof(ie,:)=[ie,enod(ie,:)];
end
```

The output `e` is *edges* and contains among other things node-pairs belonging to a certain boundary segment. Thus if you activate the option in PDEtool, <Show Edge Labels> and <Show Subdomain Labels> in boundary mode you can see how the different segments are numbered in `e` and subdomains in `t`. For our intents and purposes the rows 1,2 and 5 are the relevant ones. Row 1 and 2 includes the node numbers of the element-segment and row 5 is the edge label, i.e. which boundary segment it belongs to. For instance a list of all the convective boundaries for the heat problems can be found from

```
% Check which segments that should have convections
er = e([1 2 5],:);   % Reduced e

%conv_segments = [10 11 12]; % Choosen boundary segments
edges_conv = [];
for i = 1:size(er,2)
if ismember(er(3,i),conv_segments)
edges_conv = [edges_conv; er(1:2,i)'];
end
end
```

`edges_conv` now contains the node numbers of the node pairs that belongs to the convective part of the global boundary. From this the length of each segment L can be determined when calculating the contributions to boundary force vector and the stiffness matrix. Figure 9 shows an example of how the segment numbering and subdomains can look like.

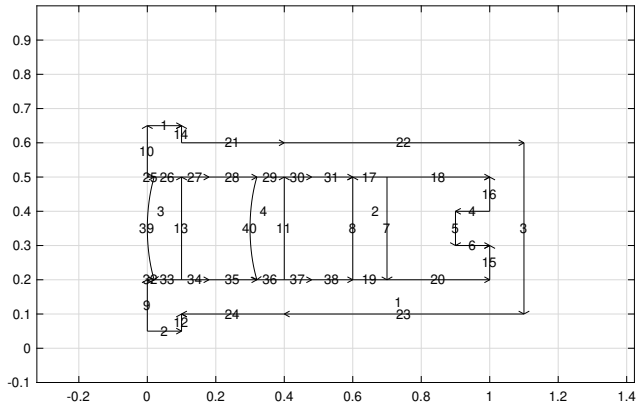

Figure 9: Example geometry of a lens with subdomains (red numbers) and edge labels (black numbers). Note that the meshing will be constrained by the edges (grey lines) so that an element is wholly within a single subdomain. Boundary segment 1 and 14 in this case corresponds to where the boundary conditions are changing at the outer boundary, e.g. 14 is isolated whereas 1 has convection.

Having made a mesh in PDE, it is trivial to refine the mesh. Therefore, it is advantageous to use a small mesh to test your code since it is much faster. When you think it is working use a finer mesh to get more accurate results.

From experience, the CALFEM functions `assem` and `insert` are very slow. They can be replaced by array indexing, as shown in the examples below

```
%  Kt = assem(edof(el,:),Kt,Kte);
indx = edof(el,2:end);
Kt(indx,indx) = Kt(indx,indx)+Kte;

% f = insert(edof(el,:),f,fe);
indx = edof(el,2:end);
f(indx) = f(indx) + fe;
```

It is always nice to present nice plots, here are some short tips for plotting.

For instance the MATLAB function `patch` can be used to generate field plots, e.g. `patch(ex',ey',eT')` where `eT` is the element temperatures obtained from `eT=extract(edof,T)` with `T` as the nodal temperatures. To plot the whole component (both sides of the symmetry cut) you can therefore write:

```
patch(ex',ey',eT')
hold on
patch(-ex',ey',eT')
```

If you use a fine mesh, the mesh lines can sometimes obscure the actual result. To remove the mesh lines from the plotting the option `'EdgeColor'` can be turned off (note that somewhere in the report the mesh used for the results should be shown), i.e. `patch(ex',ey',eT','EdgeColor','none')`.

Don't forget to turn on the colorbar and choose an illustrative color scale, e.g. `colormap(hot)` and set the axis scales to the correct proportions. Example:

```
figure()
patch(ex',ey',eT','EdgeColor','none')
title('Temperature distribution [C]')
colormap(hot);
colorbar;
xlabel('x-position [m]')
```

```
ylabel('y-position [m]')
axis equal
```

In order to compare several plots with each other it is recommended to use the same scale. By setting option `caxis([Tmin Tmax])` where `Tmin` and `Tmax` can be calculated or chosen.

To compare the deformation patterns it is preferred they are plotted on top of each other. This can be done in several ways but if `patch` is used the opacity can be set by the option `'FaceAlpha'` where 1 is completely opaque and 0 completely transparent. Example:

```
% Calculate displaced coordinates
mag = 100; % Magnification (due to small deformations)
exd = ex + mag*ed(:,1:2:end);
eyd = ey + mag*ed(:,2:2:end);

figure()
patch(ex',ey',[0 0 0],'EdgeColor','none','FaceAlpha',0.3)
hold on
patch(exd',eyd',[0 0 0],'FaceAlpha',0.3)
axis equal
title('Displacement field [Magnitude enhancement 100]')
```

where the third argument is a color triplet, i.e. `[0 0 0]` is black, so with `'FaceAlpha'=0.3` the result will be a grey-scale plot. You might have to change the magnitude to get nice plots.